

Báo cáo Đồ án 1: Triển khai hệ thống CI

Mục lục

Mục lục.....	0
I. Thành viên nhóm.....	1
II. Mở đầu.....	1
III. Giải pháp và Triển khai.....	1
3.1. Lựa chọn công cụ CI.....	1
3.2. Khởi tạo và Quản lý Repository.....	2
3.3. Cấu hình Branch Protection.....	2
3.4. Quản lý Pipeline đa nhánh.....	3
3.5. Tổng quan về xây dựng GitHub Actions CI/CD Pipeline:.....	6
3.6. Tối ưu hóa Monorepo.....	8
3.7. Bổ sung Unit Test và Tăng độ phủ.....	10
3.8. Quality Gate.....	10
3.9. Quét lỗ hổng bảo mật và chất lượng code.....	11
3.9.1. Gitleaks.....	11
3.9.2. SonarQube.....	14
3.9.3. Snyk.....	16
IV. Phụ lục.....	18
4.1. Thông tin nộp bài.....	18
4.2. Quy trình CI/CD với GitHub Actions.....	18
Cấu trúc các Workflow.....	18
Cấu hình SonarQube trong mã nguồn.....	19
Các Secrets cần cấu hình trên GitHub.....	19
Danh sách các Action được sử dụng.....	20

I. Thành viên nhóm

Đồ án được thực hiện bởi nhóm gồm 4 sinh viên:

STT	Họ và tên	MSSV
1	Nguyễn Lê Hoàng Trung	23122004
2	Lê Hoàng Minh Huy	23122033
3	Châu Văn Minh Khoa	23122035
4	Trần Tạ Quang Minh	23122042

II. Mở đầu

Đồ án này tập trung vào việc xây dựng và triển khai quy trình Continuous Integration (CI) cho dự án mã nguồn mở YAS: Yet Another Shop. Đây là một hệ thống microservice điển hình được phát triển bằng Java và hệ sinh thái Spring Boot, tích hợp nhiều công nghệ hiện đại như Kafka, Keycloak, Elasticsearch, Kubernetes, OpenTelemetry và Grafana. Mục tiêu chính của đồ án là thiết lập pipeline tự động nhằm đảm bảo chất lượng mã nguồn, tự động hóa quá trình kiểm thử, phân tích mã nguồn và build ứng dụng trước khi tích hợp vào nhánh chính.

Hiện tại, dự án YAS đã được cập nhật lên Java 25 trong các phiên bản mới hơn, thay vì Java 21 như trước đây. Đồng thời, hệ thống vẫn sử dụng Spring Boot 3.2 cùng nhiều công nghệ hỗ trợ cho kiến trúc microservice và observability.

III. Giải pháp và Triển khai

Dưới đây là chi tiết các bước triển khai dựa trên các yêu cầu cụ thể của đồ án:

3.1. Lựa chọn công cụ CI

Nhóm quyết định sử dụng **GitHub Action** để xây dựng pipeline do khả năng tích hợp tốt với hệ sinh thái mã nguồn mở.



3.2. Khởi tạo và Quản lý Repository

Nhóm đã thực hiện Fork repository từ địa chỉ gốc [nashtech-garage/yas](https://github.com/nashtech-garage/yas) về tài khoản cá nhân/nhóm để quản lý quy trình phát triển riêng.

Đây là đường dẫn tới repository của nhóm: <https://github.com/23122004/DevOps-Project01>

3.3. Cấu hình Branch Protection

Để đảm bảo tính ổn định của nhánh **main**, nhóm đã thiết lập các quy tắc bảo vệ nhánh:

The screenshot shows the GitHub 'Rulesets' configuration page for the 'main' branch. At the top, it says 'Rulesets / main' with an 'Active' status indicator. Below this, the 'Ruleset Name' is set to 'main'. The 'Enforcement status' is set to 'Active'. There is a 'Bypass list' section with a '+ Add bypass' button and a note: 'Exempt roles, teams, agents, and apps from this ruleset by adding them to the bypass list.' Below this, a box states 'Bypass list is empty'. The 'Target branches' section asks 'Which branches should be matched?' and shows a 'Branch targeting criteria' section with an 'Add target' button. Under 'Branch targeting criteria', there is a '+ Default' button and a trash icon. At the bottom, it says 'Applies to 1 target: main'.

Hình 1: Bảo vệ nhánh **main**.

Mỗi Pull Request (PR) yêu cầu ít nhất 2 đánh giá (approval) từ các thành viên khác.

-
- ☒ **Require a pull request before merging**
Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.

Hide additional settings ^

Required approvals

2 ▾

Hình 2: Ngăn chặn việc push trực tiếp vào nhánh **main** (cần mở Pull Request).

Các Pull Request phải vượt qua các bước kiểm tra CI trước khi được phép merge.

-
- ☒ **Require status checks to pass**
Choose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another ref where the checks pass.
- Hide additional settings ^
- ☐ **Require branches to be up to date before merging**
Whether pull requests targeting a matching branch must be tested with the latest code. This setting will not take effect unless at least one status check is enabled.
- ☐ **Do not require status checks on creation**
Allow repositories and branches to be created if a check would otherwise prohibit it.

Hình 3: Yêu cầu các bước kiểm tra CI phải thành công (pass) trước khi được phép merge.

3.4. Quản lý Pipeline đa nhánh

Do dự án được tổ chức theo mô hình monorepo, nhóm triển khai hệ thống CI bằng GitHub Actions với các workflow độc lập cho từng microservice.

3.4.1. Cơ chế kích hoạt và Phân quyền

Toàn bộ workflow được cấu hình để tự động hóa hoàn toàn nhưng vẫn đảm bảo tính kiểm soát:

- **Trigger dựa trên phạm vi:** Pipeline chỉ kích hoạt khi có thay đổi trong thư mục của chính service đó hoặc các file cấu hình dùng chung (như `.github/workflows/actions`).
- **Phân quyền tối thiểu:** Mỗi workflow được khai báo quyền cụ thể (`contents: read`, `checks: write`, `pull-requests: write`), đảm bảo an toàn cho repository.
- **Kích hoạt thủ công:** Cho phép kích hoạt thủ công để kiểm thử nhanh mà không cần tạo commit.

3.4.2. Quy trình CI cho các services

Đối với Frontend (Node.js - Storefront,):

- **Kiểm tra định dạng & Linting:** Sử dụng `Prettier` và `ESLint` để duy trì tiêu chuẩn code.
- **Quản lý lỗ hổng (Gitleaks, Snyk):** Tự động quét các thư viện phụ thuộc để phát hiện các lỗ hổng bảo mật nghiêm trọng (Ví dụ: Rò rỉ API Keys).
- **Phân tích Code Quality:** Tích hợp **SonarCloud** để đánh giá mã nguồn chất lượng thấp

Đối với dịch vụ Backend (Java/Maven - Tất cả services còn lại):

Dịch vụ này áp dụng mô hình Pipeline hai Jobs:

1. **Test:** Thực hiện unit test, xuất báo cáo kết quả qua `test-reporter`, đồng thời đo lường độ bao phủ mã nguồn (**JaCoCo**) với ngưỡng tối thiểu 70%. Kết quả phân tích được đẩy lên SonarCloud.
2. **Build:** Sau khi test vượt qua, hệ thống tiến hành Checkstyle và thực hiện quét bảo mật bằng **Snyk** và **OWASP Dependency Check**.

3.4.3. Đóng gói và Lưu trữ Artifact

Đối với cả hai loại dịch vụ, khi mã nguồn được merge vào `main`, pipeline sẽ tự động thực hiện các bước cuối cùng:

- **Dockerization:** Build Docker image từ Dockerfile của từng service.
- **Container Registry:** Đăng nhập và push image lên **GitHub Container Registry** (ghcr.io).

Hiện tại, bước push lên production thông qua docker này chưa được nhóm configure vì nhóm muốn tập trung vào cải thiện CI, đúng với yêu cầu của đề án.

3.4.4. Ví dụ cấu hình Pipeline

Dưới đây là minh họa cấu hình cho dịch vụ **Location** (Java), thể hiện sự tách biệt giữa các giai đoạn kiểm tra:

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
```

```
- name: Run unit tests

  run: mvn clean install -pl location -am

- name: Analyze with sonar cloud

- name: Upload JaCoCo coverage report

build:

  needs: test

  runs-on: ubuntu-latest

  steps:

    - name: Run Maven Checkstyle

    - name: OWASP Dependency Check

      run: ...

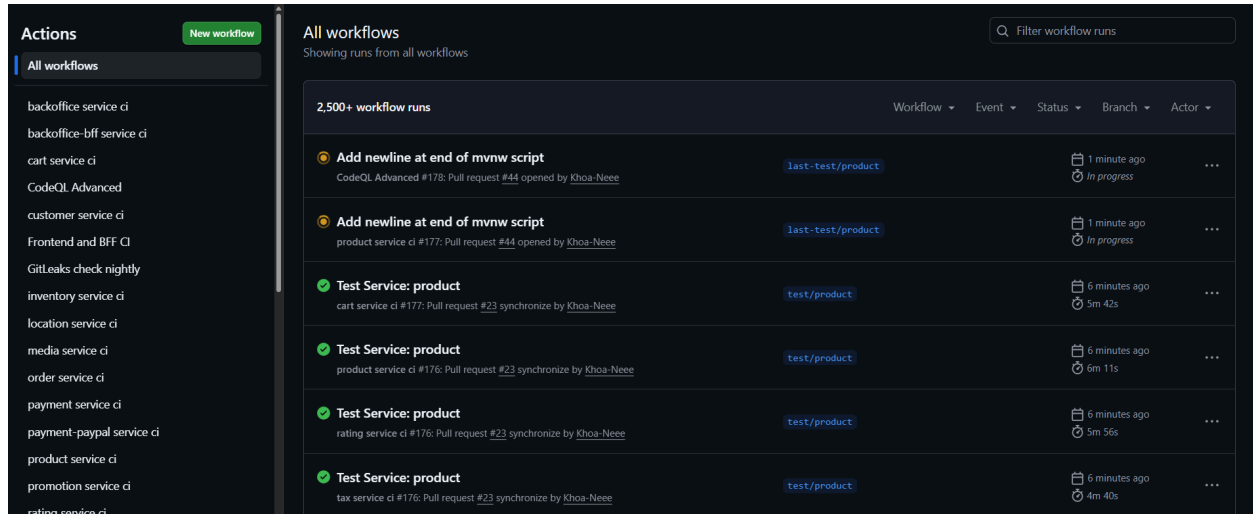
    - name: Snyk Dependency Scan

  env:

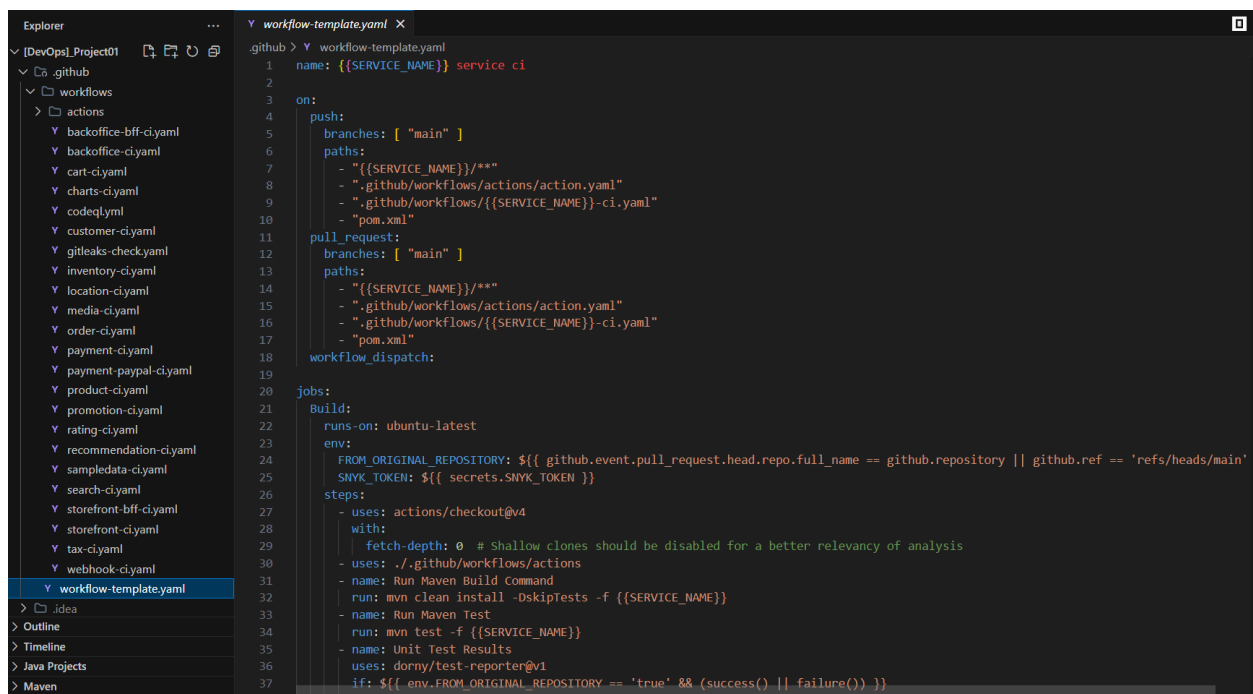
    SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
```

Việc áp dụng các công cụ như **SonarCloud**, **JaCoCo**, **Snyk** và **OWASP** vào pipeline không chỉ giúp phát hiện lỗi sớm mà còn đảm bảo các dịch vụ được triển khai luôn đạt tiêu chuẩn về cả chất lượng mã nguồn lẫn độ an toàn bảo mật. Ngoài ra, vì yêu cầu đề bài không yêu cầu chỉnh sửa lại codebase gốc, nên nhóm đã thực hiện continue-on-error trên Snyk Dependency Scan nhằm mục đích test các tính năng khác.

3.5. Tổng quan về xây dựng GitHub Actions CI/CD Pipeline:



Hình 10a: Dashboard Github Actions Pipeline



Hình 10b: Cấu hình file Workflow Github được lưu tại .github/workflows/*.yaml

Kiến trúc Pipeline của nhóm bao gồm các Phase chính:

1. Initialize & Path Filtering (Khởi tạo và Phân loại):

- Thay vì sử dụng một Pipeline duy nhất, nhóm chia nhỏ thành các Workflow riêng biệt cho từng Microservice (**tax-ci.yaml**, **cart-ci.yaml**, ...).
- Logic Build: Sử dụng tính năng **paths** của GitHub Actions để chỉ kích hoạt Pipeline khi có thay đổi trong thư mục của chính service đó hoặc các file cấu hình dùng chung (**pom.xml**, **.github/workflows/actions/**).

2. Secret Scan (Gitleaks):

- Nhóm cấu hình workflow `gitleaks-check.yaml` chạy định kỳ hàng đêm (Schedule).
- Mục tiêu: Sử dụng Gitleaks (phiên bản v8.18.4) để quét toàn bộ lịch sử commit, phát hiện kịp thời việc vô tình để lộ Credentials, Tokens hoặc API Keys.

3. Dependency Setup & Common Library:

- Hành động: Sử dụng `actions/setup-java@v4` để thiết lập JDK 21 (Temurin).
- Xử lý ưu tiên: Khi build một microservice, nhóm sử dụng cờ `-am` (also-make) trong lệnh Maven: `mvn clean install -pl [service] -am`. Lệnh này đảm bảo `common-library` luôn được tự động build và cập nhật trước khi build microservice, đảm bảo tính tương thích và toàn vẹn dữ liệu.

4. Microservices Pipeline:

- Mỗi service khi được kích hoạt sẽ trải qua chu trình:
 - + Static Code Analysis: Chạy `checkstyle:checkstyle` để đảm bảo định dạng code thống nhất.
 - + Unit Test & Reporting: Chạy các test cases và sử dụng `dorny/test-reporter` để hiển thị kết quả JUnit trực quan ngay trên giao diện GitHub.
 - + SonarCloud Integration: Phân tích chất lượng mã nguồn độc lập, đẩy dữ liệu lên SonarCloud với các tiêu chí về Code Smell, Bugs và Security Hotspots.

5. Security Scan (Snyk & OWASP):

- Snyk Dependency Scan: Được tích hợp trực tiếp vào CI của từng service để quét các lỗ hổng bảo mật trong thư viện bên thứ ba. Nhóm cấu hình quét sâu vào file `pom.xml` của từng module cụ thể.
- OWASP Dependency Check: Được chạy song song dưới dạng Docker container trong pipeline để cung cấp thêm một báo cáo bảo mật độc lập, xuất kết quả ra định dạng HTML để lưu trữ.

6. Post Actions & Quality Gate:

- Quality Gate (Coverage): Sử dụng `madrapps/jacoco-report` để đọc file báo cáo từ Jacoco. Hệ thống sẽ tự động comment kết quả độ bao phủ vào Pull Request. Nhóm thiết lập ngưỡng Quality Gate là 70%; nếu độ bao phủ thấp hơn, pipeline sẽ đánh dấu không đạt.
- Dockerization: Khi mã nguồn được merge vào nhánh `main`, phase cuối cùng sẽ build Docker image và push lên GitHub Container Registry (ghcr.io), sẵn sàng deploy.
- Thông báo trạng thái: Kết quả của từng Phase (Build, Test, Scan) được hiển thị trực tiếp trong mục "Checks" của GitHub PR, giúp tác giả biết chính xác bước nào bị lỗi để khắc phục.

3.6. Tối ưu hóa Monorepo

Do dự án sử dụng mô hình monorepo, quy trình CI/CD bằng GitHub Actions được cấu hình (thông qua cơ chế paths filter) để chỉ kích hoạt workflow cho các service cụ thể (ví dụ:

Media-service, Product-service...) khi có thay đổi trong thư mục cấu trúc tương ứng của service đó. Điều này giúp tiết kiệm tối đa tài nguyên chạy CI và thời gian build hệ thống.

Tuy nhiên, hệ thống vẫn đảm bảo tính an toàn bằng cách nhận diện các file cấu hình chung. Khi có chỉnh sửa trên các tệp dùng chung của dự án (ví dụ: pom.xml ở thư mục gốc hoặc các file cấu hình action chung trong actions), GitHub Actions sẽ nhận diện và chạy đồng loạt workflow để Build và Test trên toàn bộ các service:

2,500+ workflow runs		Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
✔	Feat: Merge all unit test storefront service ci #172: Pull request #43 synchronize by BrdH7940	feat/merging		24 minutes ago 5m 18s	...	
✔	Feat: Merge all unit test media service ci #175: Pull request #43 synchronize by BrdH7940	feat/merging		24 minutes ago 8m 4s	...	
✔	Feat: Merge all unit test search service ci #175: Pull request #43 synchronize by BrdH7940	feat/merging		24 minutes ago 8m 29s	...	
✔	Feat: Merge all unit test CodeQL Advanced #176: Pull request #43 synchronize by BrdH7940	feat/merging		24 minutes ago 7m 56s	...	
✔	Feat: Merge all unit test product service ci #175: Pull request #43 synchronize by BrdH7940	feat/merging		24 minutes ago 7m 47s	...	
✔	Feat: Merge all unit test GitLeaks check nightly #28: Pull request #43 synchronize by BrdH7940	feat/merging		24 minutes ago 5m 4s	...	
✔	Feat: Merge all unit test sampledata service ci #175: Pull request #43 synchronize by BrdH7940	feat/merging		24 minutes ago 7m 25s	...	
✔	Feat: Merge all unit test backoffice-bff service ci #175: Pull request #43 synchronize by BrdH7940	feat/merging		24 minutes ago 6m 37s	...	

Hình 11: Nhiều GitHub Action Workflows được kích hoạt cùng lúc khi có chỉnh sửa trên file cấu hình chung (pom.xml hoặc action chung).

Ngược lại, nếu chỉ có thay đổi mã nguồn hoặc cấu hình cục bộ trong thư mục của một service cụ thể nào đó thì GitHub Actions sẽ chỉ đánh giá và chạy riêng rẽ workflow của service đó, không cần Build và Test lại toàn bộ hệ thống:

2,500+ workflow runs		Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
🟡	Add newline at end of mvnw script CodeQL Advanced #178: Pull request #44 opened by Khoa-Neee	last-test/product		1 minute ago In progress	...	
🟡	Add newline at end of mvnw script product service ci #177: Pull request #44 opened by Khoa-Neee	last-test/product		1 minute ago In progress	...	

Hình 12a: Github Actions khi có chỉnh sửa trên service product

2,500+ workflow runs			Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
●	Add newline at end of mvnw script	last-test/product	now	Queued	...	payment service ci #177: Pull request #44 synchronize by Khoa-Neee	
●	Add newline at end of mvnw script	last-test/product	now	In progress	...	product service ci #178: Pull request #44 synchronize by Khoa-Neee	
●	Add newline at end of mvnw script	last-test/product	now	Queued	...	CodeQL Advanced #179: Pull request #44 synchronize by Khoa-Neee	

Hình 12b: Github Actions khi có chỉnh sửa trên service product và service payment

3.7. Bổ sung Unit Test và Tăng độ phủ

Nhóm đã tạo các nhánh riêng biệt cho từng service như Media, Product, Cart,... để bổ sung thêm các unit test, giúp cải thiện độ bao phủ. Sau khi tạo Unit Test cho từng service, nhóm mở PR cho từng service. Dưới đây là kết quả chạy các Pipeline trên Github:

22 Open ✓ 22 Closed		Author ▾	Label ▾	Projects ▾	Milestones ▾	Reviews ▾	Assignee ▾	Sort ▾
Feat: Merge all unit test ✓	#43 opened yesterday by htrung1105	Collaborator	Approved	79				
Test Service: webhook ✓	#42 opened yesterday by htrung1105	Collaborator	Review required	19				
Test Service: tax ✓	#41 opened yesterday by htrung1105	Collaborator	Review required	17				
Test Service: search ✗	#40 opened yesterday by htrung1105	Collaborator	Review required	17				
Test Service: sampledata ✗	#39 opened yesterday by htrung1105	Collaborator	Review required	21				
Test Service: recommendation ✓	#38 opened yesterday by htrung1105	Collaborator	Review required	23				

Hình 13: Các PR cho từng service trên Github

3.8. Quality Gate

Pipeline được điều chỉnh chỉ cho phép merge khi độ phủ của testcase trên mức tối thiểu **70%**. Cụ thể, nhóm sử dụng plugin JaCoCo của Maven để đo lường và thu thập dữ liệu độ phủ khi các Unit Test được chạy.

```
- name: Add coverage report to PR
  uses: madrapps/jacoco-report@v1.6.1
  if: ${{ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
  with:
    paths: ${{github.workspace}}/{{SERVICE_NAME}}/target/site/jacoco/jacoco.xml
    token: ${{secrets.GITHUB_TOKEN}}
    min-coverage-overall: 70
```

```
min-coverage-changed-files: 70
title: '{{SERVICE_NAME}} Coverage Report'
pass-coverage-on-overall-minimum: true
update-comment: true
```

Do đó, Pipeline chỉ được coi là thành công khi độ phủ dòng lệnh đạt từ 70% trở lên; ngược lại, pipeline sẽ bị fail và không thể tiếp tục quy trình merge.



3.9. Quét lỗ hổng bảo mật và chất lượng code

Tích hợp các công cụ chuyên dụng vào pipeline:

- **Gitleaks:** Quét các thông tin nhạy cảm (secrets) bị lộ trong code.
- **SonarQube:** Phân tích chất lượng mã nguồn và các chỉ số kỹ thuật.
- **Snyk:** Kiểm tra các lỗ hổng bảo mật trong các thư viện phụ thuộc.

3.9.1. Gitleaks

Quá trình quét được cấu hình trong workflow GitLeaks check nightly thay vì chạy chung nhánh CI:

```

hub > workflows > ! gitleaks-check.yaml
1  name: Gitleaks check nightly
2  on:
3    workflow_dispatch:
4    schedule:
5      - cron: "0 0 * * *"
6  jobs:
7    check:
8      runs-on: ubuntu-latest
9      steps:
10     - name: Checkout
11       uses: actions/checkout@v4
12       with:
13         fetch-depth: 0 # Shallow clones should be disabled for a better relevancy of analysis
14     - name: Gitleaks check
15       run: |
16         docker pull zricethezav/gitleaks:v8.18.4
17         docker run --rm -v ${GITHUB_WORKSPACE}:/work -w /work zricethezav/gitleaks:v8.18.4 detect --sou

```

Các bước chính trong stage Secret Scan (Gitleaks):

1. Checkout mã nguồn Workflow sử dụng:

Workflow sử dụng `actions/checkout@v4` với tham số `fetch-depth: 0` để đảm bảo hệ thống kéo đầy đủ mã nguồn và lịch sử commit về workspace nhằm phục vụ cho quá trình phân tích trọn vẹn nhất.

2. Thực thi quét secret

```

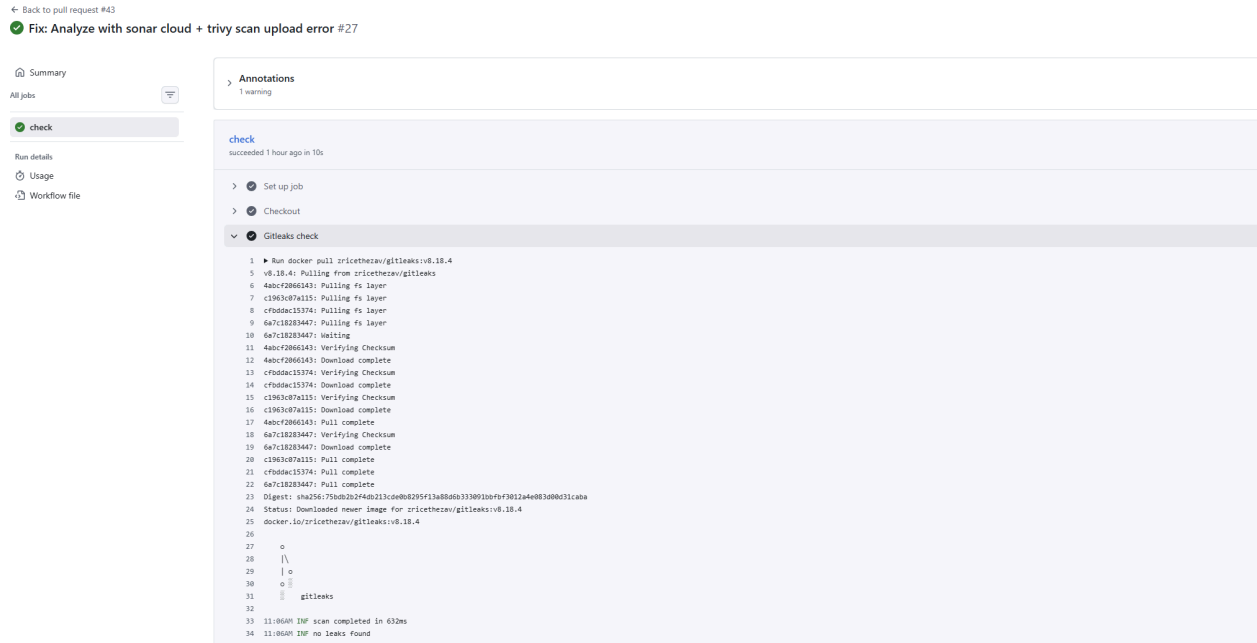
docker run --rm -v ${GITHUB_WORKSPACE}:/work -w /work \
  zricethezav/gitleaks:v8.18.4 detect \
  --source="." \
  --config="/work/gitleaks.toml" \
  --verbose \
  --no-git

```

Các tham số:

- `-v ${GITHUB_WORKSPACE}:/work`: Mount mã nguồn từ máy chủ GitHub Runner vào container để quét.
- `--source="."`: Quét toàn bộ thư mục làm việc hiện tại.
- `--config="/work/gitleaks.toml"`: Áp dụng cấu hình và allowlist đặc thù của dự án.
- `--no-git`: Quét trực tiếp nội dung file hiện tại thay vì lịch sử commit git, giúp tối ưu thời gian chạy.
- `--verbose`: Xuất log chi tiết giúp lập trình viên dễ dàng truy vết vị trí lỗi trên giao diện GitHub Actions.

3. Lưu trữ kết quả scan



Hình 14a: Gitleaks Scan Warnings khi không có leaked secret

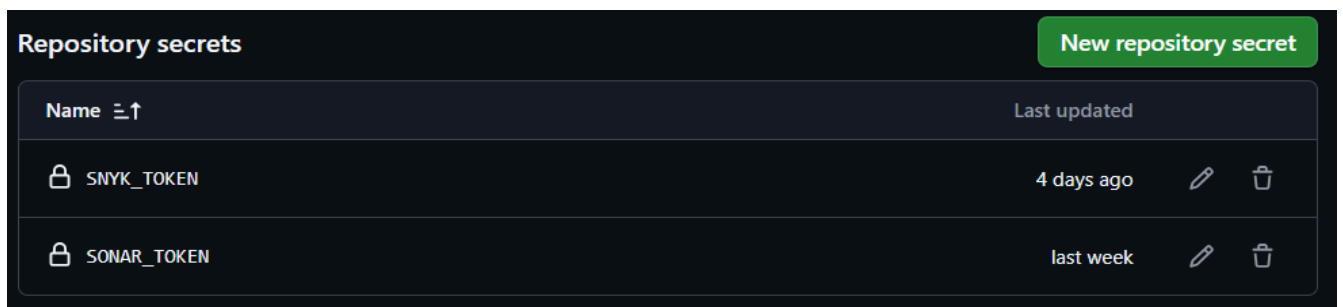
3.9.2. SonarQube

SonarQube được sử dụng để phân tích chất lượng mã nguồn, theo dõi bug, code smell và security issues cho từng microservice.

Để tích hợp SonarQube vào pipeline, nhóm thực hiện cấu hình cả trong [Github Action](#) và các file [pom.xml](#).

1. Cấu hình môi trường SonarQube trong Github Actions

Đầu tiên ta cần khai báo secrets token xác thực để Github Action kết nối với SonarQube



2. Thực thi phân tích mã nguồn trong pipeline

```
# File: .github/workflows/*-ci.yaml
- name: Analyze with sonar cloud
  if: ${ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }}
  run: mvn org.sonarsource.scanner.maven:sonar-maven-plugin:sonar
-Dsonar.organization=23122004 -Dsonar.projectKey=23122004_DevOps-Project01
-Dsonar.host.url=https://sonarcloud.io
-Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml
-Dsonar.qualitygate.wait=true
```

Với config này, tất cả microservice được phân tích trong cùng một project trên SonarQube.

3. Cấu hình properties trong pom.xml

Tại file **pom.xml** ở root project:

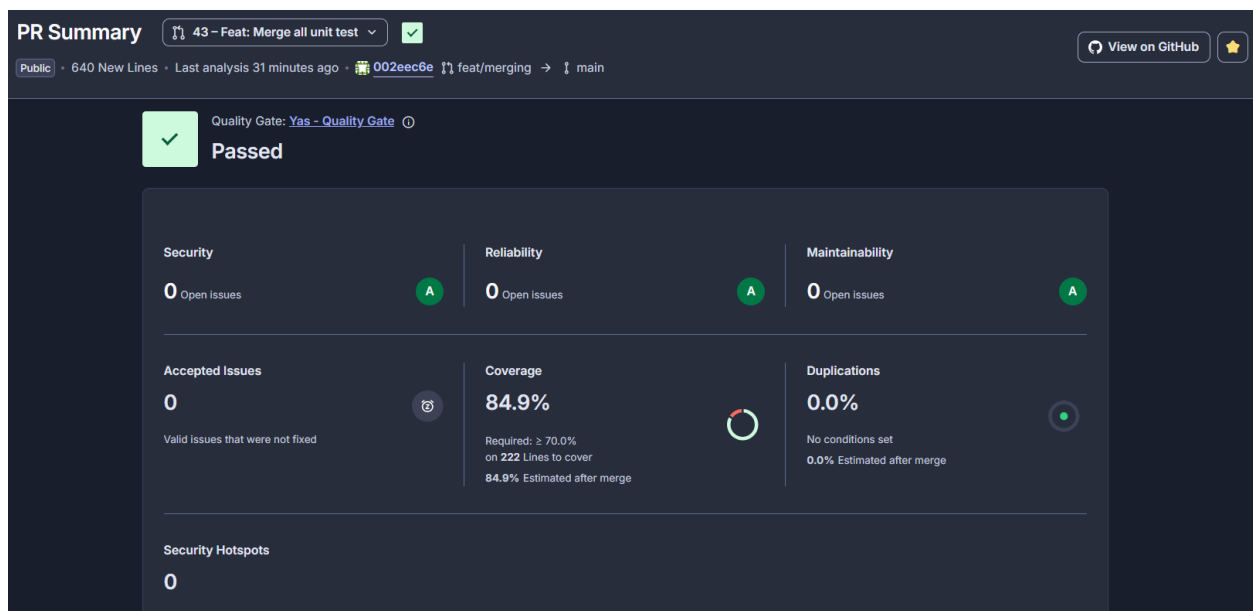
```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>${jacoco-maven-plugin.version}</version>
  <executions>
    <!-- Khởi tạo agent để theo dõi test -->
    <execution>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
    <!-- Tổng hợp và xuất báo cáo sau khi chạy test xong -->
    <execution>
      <id>report</id>
      <phase>verify</phase>
      <goals>
        <goal>report</goal>
      </goals>
    </execution>
  </executions>
  <!-- Loại trừ các file không cần tính coverage (như Application,
config, exceptions...) -->
  <configuration>
```

```

<excludes>
  <exclude>com/yas/**/*Application.class</exclude>
  <exclude>com/yas/**/config/**</exclude>
  <exclude>com/yas/**/exception/**</exclude>
  <exclude>com/yas/**/constants/**</exclude>
</excludes>
</configuration>
</plugin>

```

Hình ảnh kết quả trên SonarQube:



Hình 15: Kết quả đo Coverage tất cả unit-test mới của các service trên SonarQube

3.9.3. Snyk

Stage **Security Scan (Snyk - Root)** được sử dụng để quét lỗ hổng bảo mật trong các dependency của toàn bộ monorepo Maven. Thay vì quét gộp toàn bộ monorepo bằng một stage chung, quá trình quét mã nguồn độc hại bằng Snyk được phân tán và tích hợp trực tiếp vào workflow CI của từng service (ví dụ: product, storefront). Cách này giúp Snyk quét độ an toàn của các thư viện phụ thuộc (dependency) một cách chính xác theo đặc thù ngôn ngữ của từng service (chẳng hạn dùng Maven cho backend và npm cho frontend). Dưới đây là một ví dụ minh họa step Snyk trong file cấu hình:

```
- name: Snyk Dependency Scan
```

```

if: ${ env.FROM_ORIGINAL_REPOSITORY == 'true' && env.SNYK_TOKEN != '' }}
uses: snyk/actions/maven@master
env:
  SNYK_TOKEN: ${ secrets.SNYK_TOKEN }}
with:
  args: --file=product/pom.xml --severity-threshold=high

```

Các thành phần chính trong cấu hình:

- `if: ${ env.FROM_ORIGINAL_REPOSITORY == 'true' && env.SNYK_TOKEN != '' }}`: Điều kiện thực thi, đảm bảo bước này chỉ chạy trên repository gốc (không chạy trên các bản fork của cộng đồng) và chỉ chạy khi có cài đặt token bảo mật (`SNYK_TOKEN`).
- `uses: snyk/actions/maven@master`: Sử dụng bộ công cụ GitHub Action chính thức của Snyk dành riêng cho Maven.
- `env.SNYK_TOKEN`: Truyền biến môi trường an toàn chứa token xác thực kết nối với hệ thống Snyk.
- `--file=...`: Chỉ quét chính xác file quản lý thư viện của service đang build (tránh quét nhầm toàn bộ monorepo).
- `--severity-threshold=high`: Chỉ cảnh báo và báo lỗi với các lỗ hổng bảo mật có mức độ Nghiêm trọng (High) trở lên, giúp giảm nhiễu cho luồng CI.

Cấu hình phân mảnh này giúp cải thiện đáng kể tốc độ build, cô lập lỗi bảo mật của thư viện nội bộ theo từng service và tránh làm gián đoạn toàn bộ dự án nếu một service đơn nhánh gặp vấn đề.

Hình ảnh kết quả trên Snyk:

Project	Imported	Tested	Issues ↓
23122004/DevOps-Project01			157 C, 481 H, 361 M, 231 L
payment/pom.xml	21 minutes ago	21 minutes ago	9 C, 25 H, 27 M, 5 L
payment-paypal/pom.xml	21 minutes ago	20 minutes ago	9 C, 25 H, 27 M, 5 L
customer/pom.xml	21 minutes ago	21 minutes ago	8 C, 24 H, 27 M, 5 L
order/pom.xml	21 minutes ago	21 minutes ago	8 C, 23 H, 26 M, 5 L
common-library/pom.xml	21 minutes ago	20 minutes ago	8 C, 19 H, 9 M, 5 L
recommendation/pom.xml	21 minutes ago	21 minutes ago	7 C, 23 H, 9 M, 6 L
webhook/pom.xml	21 minutes ago	20 minutes ago	7 C, 19 H, 9 M, 5 L
search/pom.xml	21 minutes ago	20 minutes ago	7 C, 18 H, 9 M, 6 L
automation-ui/storefront/pom.xml	21 minutes ago	20 minutes ago	6 C, 27 H, 20 M, 13 L

Hình 16a: Tổng quan các service trên Snyk

IV. Phụ lục

4.1. Thông tin nộp bài

- Link Repository: [DevOps-Project01](#)

4.2. Quy trình CI/CD với GitHub Actions

Sử dụng GitHub Actions làm nền tảng CI/CD chính, tận dụng mô hình Matrix/Multi-workflow để quản lý việc Build, Test và Security Scanning cho kiến trúc Microservices.

Cấu trúc các Workflow

Dự án được cấu hình với các file workflow riêng biệt trong thư mục `.github/workflows/`, giúp tối ưu hóa thời gian chạy pipeline bằng cách chỉ kích hoạt build những service có thay đổi (Path-based triggers).

Các bước chính trong quy trình CI (Ví dụ cho một microservice):

1. **Trigger:** Tự động kích hoạt khi có `push` hoặc `pull_request` vào nhánh `main` có thay đổi trong thư mục của service đó.
2. **Environment Setup:** Sử dụng Java 21 (Temurin) làm môi trường thực thi tiêu chuẩn.
3. **Build & Test:**
 - Sử dụng Maven để build service: `mvn clean install -pl [service] -am`
 - Kiểm tra coding convention bằng Checkstyle.
4. **Security Scanning (Tích hợp đa tầng):**
 - Gitleaks: Quét lộ bí mật (secrets/tokens) định kỳ hàng đêm (Nightly) sử dụng image `zricethezav/gitleaks:v8.18.4`
 - Snyk: Phân tích lỗ hổng trong các thư viện dependency (Snyk Dependency Scan).
 - OWASP Dependency Check: Bổ sung thêm một lớp kiểm tra lỗ hổng bảo mật cho các thành phần phần mềm.
5. **Quality Gate:**
 - SonarCloud: Phân tích chất lượng mã nguồn với Organization `23122004` và Project Key `23122004_DevOps-Project01`
 - JaCoCo: Thu thập độ bao phủ mã nguồn (Coverage), yêu cầu tối thiểu 70% để vượt qua Quality Gate.
6. **Containerization:** Nếu build thành công trên nhánh `main`, hệ thống tự động build

Docker Image và push lên GitHub Container Registry (ghcr.io).

Cấu hình SonarQube trong mã nguồn

Project sử dụng cả `pom.xml` ở root directory:

```
<properties>
  <sonar.organization>23122004</sonar.organization>
  <sonar.host.url>https://sonarcloud.io</sonar.host.url>
  <sonar.projectKey>23122004_DevOps-Project01</sonar.projectKey>
</properties>
```

Thông tin định danh dự án trên SonarCloud được thống nhất để quản lý tập trung:

- Organization: 23122004
- Host URL: https://sonarcloud.io
- Project Key: 23122004_DevOps-Project01 (Dùng chung hoặc phân tách theo service tùy theo lệnh gọi Maven).

Các Secrets cần cấu hình trên GitHub

Để các workflow hoạt động, nhóm đã thiết lập các Secret sau trong Repository:

- **SONAR_TOKEN:** Xác thực với SonarCloud.
- **SNYK_TOKEN:** Xác thực với Snyk CLI.
- **GITHUB_TOKEN:** Token mặc định của GitHub dùng để push image lên GHCR và viết comment báo cáo Coverage vào Pull Request.

Danh sách các Action tiêu chuẩn được sử dụng

Hệ thống tận dụng các Action cộng đồng mạnh mẽ để tối ưu quy trình:

Action	Mục đích
actions/checkout@v4	Kiểm xuất mã nguồn với <code>fetch-depth: 0</code> cho SonarCloud.
actions/setup-java@v4	Khởi tạo môi trường JDK 21.

snyk/actions/maven	Quét bảo mật dependency mức độ High/Critical.
madrapps/jacoco-report	Tự động cập nhật báo cáo Coverage vào PR comment.
docker/build-push-action@v6	Build và đẩy Docker Image lên registry chuyên nghiệp.
dorny/test-reporter	Tổng hợp và hiển thị kết quả Unit Test trực quan.

Các plugin trên đóng vai trò nền tảng cho toàn bộ quy trình CI/CD, giúp tự động hóa việc build, test, security scanning và publish kết quả pipeline.